PORTFOLIO

신입 게임 서버 프로그래머
우상훈입니다.

CONTACT

+82 010 5920 3386
asm1133@naver.com

CONTENTS

01

프로필

02

졸업작품 이방인(Outlander)

- 서버 구조
- 서버 기반 기술
- 전술 AI와 2D 시뮬레이터
- 계정·데이터 처리
- 문제 해결

03

...

04

...

01 프로필



About

우상훈 (Woo, Sanghoon)

2000.04.20

asm1133@naver.com

010-5920-3386

기술

언어	C, C++20
서버	IOCP, Multi-thread
클라이언트	Win32 API, OpenGL
기타	MSSQL, MySQL, Git

학력사항

2027.02 한국공학대학교 게임공학과 졸업예정

2019.02 진건고등학교 졸업

활동

2024.09 – 2024.12

학과 학술 소모임 WARP / C++ 멘토 활동

수상

2026학년도 한국공학대학교 게임공학과
졸업작품 총장상 수상

프로젝트

2024.09 – 2026.06

졸업작품 <이방인(Outlander)> 개발

2025.05 – 2025.06

Win32 API 기반의 게임 <헤드사커> 모작



이방인(Outlander)

2026 한국공학대학교 게임공학과 졸업작품 총장상 수상

장르	4인 협동 판타지 무쌍 액션
기획 의도	다수의 적을 상대하며 일당백 액션을 체험하는 판타지 배경 멀티플레이 게임
개발 기간	2024.09 – 2026.06
개발 인원	3인
사용 기술	C++20, IOCP, SQL Server LocalDB
핵심 연구 과제	<Strategic AI Enemy> 단순히 숫자만 많을 뿐인 적보다, 진형을 갖추고 전술을 활용하는 적이 몰입감을 높여줄 것이라 판단해 설정
담당 역할	<서버 프로그래밍> ServerEngine, LobbyServer, RoomServer 네트워크 · 작업 처리 · 인증 · 데이터 저장

서버 구조 / 서버 구성과 게임 입장 흐름



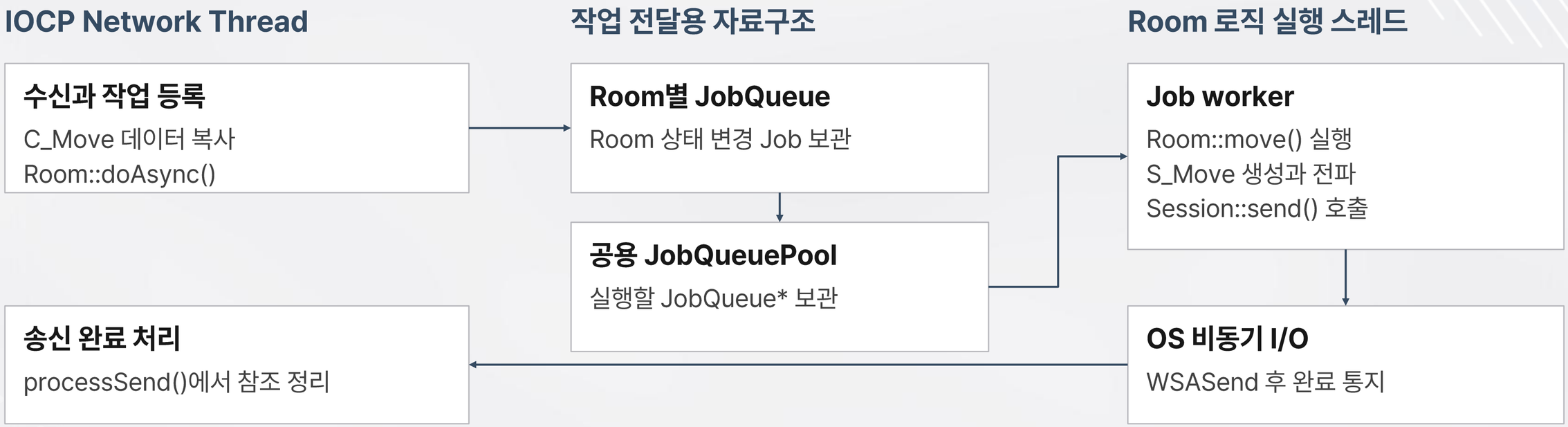
로비 · 룸 서버 간 직접 소켓 연결 없습니다.
RoomServer가 티켓의 서명과 유효를 확인합니다.

ServerEngine

두 서버가 링크하는 공통 정적 라이브러리

IOCP / 세션 / 송신 버퍼 / JobQueue / 메모리 풀 / DB

서버 구조 / 패킷 수신부터 Room 작업 실행까지



등록과 실행의 구분

Room마다 고정 스레드를 두지 않고 worker를 공유합니다.
IOCP는 Job을 등록하고 worker는 Room 로직을 실행합니다.
송신 요청은 호출 스레드, 완료 처리는 IOCP가 담당합니다.

서버 기반 기술 / Job과 JobQueue의 작업 등록

ServerEngine/Job.hpp 실제 소스 발췌

```
 7 using CallbackType = std::function<void()>;
 9 class Job {
10 public:
11   Job(CallbackType&& callback) : callback_(std::move(callback)) {}
     12–20행 생략
21   void execute() { callback_(); }
23 private:
24   CallbackType callback_;
25 };
```

ServerEngine/JobQueue.hpp 실제 소스 발췌

```
22   void doAsync(CallbackType&& callback) {
23     push(ObjectPool<Job>::pop(std::move(callback)));
24   }
```

Job 실행할 콜백 보관

콜백을 저장한 뒤 execute()에서 실행합니다.
작업의 등록과 실행 시점을 분리합니다.

doAsync() Job 생성과 전달

ObjectPool에서 얻은 Job을 Room 큐에 넣습니다.
콜백은 worke가 실행합니다.

보조 기술과 객체 수명

ObjectPool과 MemoryPool로 재사용합니다.
raw pointer 대상의 수명은별도로 보장해야 합니다.

서버 기반 기술 / JobQueue의 중복 등록 방지와 잔량 처리

등록 전 작업 수가 0이면 공용 풀에 등록

ServerEngine/JobQueue.cpp

```
5  void JobQueue::push(Job* job) {
6    const int32 prevCnt = jobCount_.fetch_add(1);
7    queue_.enqueue(job);
9    if (prevCnt == 0) {
10     JobQueuePool::push(this);
11   }
12 }
```

jobCount_는 실행 중인 작업까지 포함합니다.
추가 Job이 들어와도 기존 작업이 남아 있으면
같은 큐를 공용 풀에 다시 등록하지 않습니다.

실행을 마친 작업 수를 차감

ServerEngine/JobQueue.cpp

```
24     const int32 bulkSize = 100;
25     auto jobs = std::vector<Job*>(bulkSize);
26     auto jobCount = static_cast<int32>
   (queue_.try_dequeue_bulk(jobs.begin(), bulkSize));
28     for (int32 i = 0; i < jobCount; ++i) {
29       jobs[i]->execute();
30       ObjectPool<Job>::push(jobs[i]);
31     }
   32–36행 생략
37     const int32 remaining =
   jobCount_.fetch_sub(jobCount) - jobCount;
   38–41행 생략
42     if (remaining <= 0) break;
```

실행 중 새 Job이 추가되면 잔량에 반영합니다.
잔량이 양수이면 다음 묶음을 처리합니다.
100개는 묶음 크기이며 실행 시간 제한이 아닙니다.

서버 기반 기술 / JobQueuePool과 worker 연결

ServerEngine/JobQueuePool.hpp 실제 소스 발췌

```
10 class JobQueuePool {
11 public:
12   static void push(JobQueue* jobQueue) {
13     queue_.enqueue(jobQueue);
14   }
16   static JobQueue* pop() {
17     JobQueue* jobQueue;
19     if (queue_.try_dequeue(jobQueue)) {
20       return jobQueue;
21     }
22     else {
23       return nullptr;
24     }
25   }
27 private:
28   static ccqueue<JobQueue*> queue_;
29 };
```

Job*가 아닌 JobQueue*를 저장합니다.

RoomServer/RoomServer.cpp 실제 소스 발췌

```
31 void DoJob() {
32   while (true) {
33     auto jq = JobQueuePool::pop();
34     if (!jq) continue;
36     jq->execute();
37   }
38 }
```

ServerEngine/types.hpp 실제 소스 발췌

```
17 template<class T>
18 using ccqueue = moodycamel::ConcurrentQueue<T>;
```

동시 접근을 위한 자료구조 선택

여러 등록 스레드와 worker가 큐를 공유합니다.
lock-free queue와 묶음 꺼내기를 사용합니다.
생산자 사이의 전체 순서는 보장하지 않습니다.

서버 기반 기술 / JobTimer의 예약 작업 전달

RoomServer/JobTimer.cpp

```
11 void JobTimer::addJobAt(HighResolutionClock::time_point
   executionTime, uint32 roomId, Job* job) {
12   auto jobData = ObjectPool<JobData>::pop(roomId, job);
14   std::lock_guard<std::mutex> lock(jobTimerMtx_);
15   timerQueue_.push({executionTime, jobData});
16 }
```

시간순 예약 관리

실행 시각과 Room ID, Job을 저장합니다.
priority_queue를 mutex로 보호하고
가장 이른 예약부터 만료 여부를 확인합니다.

JobTimer
잠금 안에서 만료 예약 추출

→

Room의 JobQueue
잠금 해제 후 postJob() 호출

→

Job worker
공용 풀에서 큐를 꺼내 실행

전달 실패와 실행 시각

목적지 Room이 없으면 Job을 반환합니다. 예약 시각이 지나도 worker 처리까지 대기할 수 있습니다.

서버 기반 기술 / SendBuffer의 청크 할당과 공유

스레드별 청크
64KB 청크에서 공간 할당
남은 공간 부족 시 청크 교체

→

SendBuffer
패킷 데이터 구간 참조
청크를 shared_ptr로 보유

→

각 Session
같은 버퍼를 송신 큐에 보관
세션별로 비동기 송신

RoomServer/Room.cpp

```
2345 void Room::broadcast(const std::shared_ptr<SendBuffer>&
     sendBuffer) {
2346   for (auto session : sessions_) {
2347     session->send(sendBuffer);
2348   }
2349 }
```

ServerEngine/SendBuffer.hpp

```
45 private:
46     std::shared_ptr<SendBufferChunk> owner_;
47     byte* buffer_;
48   uint32 allocSize_;
```

한 번 만든 패킷을 수신자별로 복제하지 않고 공유합니다.

운영체제까지의 무복사 송신을 의미하지는 않습니다.

서버 기반 기술 / 비동기 송신과 메모리 수명

예시: 생성 측 참조는 해제했고 A·B 두 세션만 SendBuffer를 보유한 상태

A·B 송신 중
두 세션이 버퍼 참조 유지

→

A 송신 완료
A 참조 해제, B 참조 유지

→

B 송신 완료
마지막 버퍼 참조 해제
SendBuffer를 풀에 반환

ServerEngine/IoEvent.hpp

```
90    std::vector<std::shared_ptr<SendBuffer>>
    sendBuffers_;
```

ServerEngine/Session.cpp

```
159 }
161 void Session::processSend(int32 numBytes) {
162   sendEv_.setOwner(nullptr);  // release reference
163   sendEv_.sendBuffers_.clear();
```

청크 반환은 별도 조건

SendBuffer가 사라져도 다른 버퍼나
TLS가 청크를 참조하면 청크는 남습니다.

모든 청크 참조가 해제되면 재사용 큐로
반환하고, 종료 단계에서는 실제 해제합니다.

전술 AI와 2D 시뮬레이터

계정·데이터 처리 / 계정 인증과 DB 전용 스레드

IOCP worker
입력 검증과 값 복사
세션 shared_ptr 캡처

→

DBExecutor
전용 스레드에서 ODBC 실행
계정 조회와 비밀번호 확인

→

로그인 결과
인증 상태 기록
세션에 응답 패킷 전송

LobbyServer/PacketManager.cpp

```
212   auto self = std::static_pointer_cast<GameSession>( session->shared_from_this() );
214   DBExecutor::post(
215     [self,
216      loginId = std::string( pkt->loginId, idLen ),
217      password = std::string( pkt->password, pwLen )]() {
218       int64 accountId = 0;
219       std::wstring nickname;
220       AccountResult result = loginAccount( loginId, password, accountId, nickname );
```

DB 작업을 분리한 이유

ODBC는 호출한 스레드를 대기시킵니다.
IOCP 수신 처리와 Room 작업이
DB 응답을 직접 기다리지 않게 분리했습니다.

저장과 전송의 한계

DB에는 salt와 해시를 저장합니다.
현재 전송에는 TLS를 적용하지 않았고
비밀번호 저장에는 단순 SHA-256을 씁니다.

계정·데이터 처리 / 입장 티켓으로 인증 정보 전달

LobbyServer
계정·방 코드·만료 시각을
HMAC-SHA256으로 서명

→

Client
룸 주소와 티켓 수신
C_Enter로 티켓 제출

→

RoomServer
MAC·버전·시각·형식 검증
계정 등록 후 Room 입장

신뢰할 정보의 출처

클라이언트가 전달한 계정 ID를
그대로 신뢰하지 않습니다.
검증한 티켓의 계정과 방 코드를 사용합니다.

입장 전 요청과 중복 접속

C_Enter 인증 전에는 다른 요청을 차단합니다.
AccountRegistry로 같은 RoomServer 안의
계정 중복 접속을 제한합니다.

계정·데이터 처리 / 인벤토리 조회·저장과 실패 처리

Room 작업
입장 후 로드 요청
진행 중 DB 작업 수 증가

→

DBExecutor
계정별 슬롯 조회
결과를 값으로 전달

→

Room 작업
doAsync()로 결과 반영
인벤토리 스냅샷 전송

조회 결과를 구분

Ok: 저장된 슬롯 적용
NewAccount: 스타터 지급 후 저장
DbError: 빈 상태 유지, 저장 금지

RoomServer/Room.cpp

```
2085    if (!session->inventoryLoaded()) {
2086      return;
2087    }
```

저장 기준

계정 ID와 슬롯 번호를 키로 사용합니다.
빈 슬롯도 저장하여 신규 계정과 구분합니다.
트랜잭션 안에서 기존 행 삭제 후 전체 저장

저장 실패 시 즉시 재시도하지 않습니다.
현재는 다음 변경 시 저장을 다시 시도합니다.
DB 완료 전에는 Room 제거를 지연합니다.

문제 해결 / 수신 버퍼 정리 후 패킷 처리 중단

증상

일정 시간 플레이 후 Size: 0, Type: 0
패킷 로그가 반복되고 수신 처리가 멈췄습니다.

원인

readPos_를 먼저 0으로 바꾸면서
남은 데이터 크기를 잘못 계산했습니다.
이미 처리한 영역까지 다시 읽었습니다.

ServerEngine/RecvBuffer.hpp

```
40   void clean() {
41     int32 remaining = dataSize();
     42–49행 생략
50       if (freeSize() < bufferSize_) {
51         memcpy(&buffer_[0], &buffer_[readPos_], remaining);
52         readPos_ = 0;
53         writePos_ = remaining;
54       }
55     }
56   }
```

위치 계산 수정

커서를 바꾸기 전에 remaining을 저장하고
복사한 크기로 writePos_를 설정합니다.

남은 검증 과제

최소 패킷 크기 검사는 현재 주석 상태입니다.
비정상 헤더 차단과 회귀 테스트가
추가로 필요합니다.

문제 해결 / 로비 방 조회와 제거의 수명 경쟁

마지막 멤버의 퇴장과 다른 클라이언트의 참가가 겹치면
조회한 방이 사용되기 전에 해제될 수 있었습니다.

LobbyServer/LobbyManager.cpp

```
24 std::shared_ptr<LobbyRoom> LobbyManager::findRoom( const
   std::string& code ) {
25   std::shared_lock lock( mutex_ );
26   auto it = rooms_.find( code );
27   return it != rooms_.end() ? it->second : nullptr;
28 }
```

LobbyServer/LobbyRoom.cpp

```
 9 bool LobbyRoom::enter( const std::shared_ptr<GameSession>& session
   ) {
10   std::lock_guard lock( mutex_ );
   11–12행 생략
13   if ( closed_ ) {
14     return false;
15   }
```

객체 수명 보장

findRoom()은 shared_ptr 복사본을 반환합니다.
맵에서 제거해도 호출 측이 참조하는 동안
방 객체의 수명을 유지합니다.

논리적으로 닫힌 방의 입장 차단

마지막 퇴장 시 잠금 안에서 closed_ 설정
enter()도 같은 잠금 안에서 closed_ 확인
객체 생존과 입장 가능 여부를 따로 다룹니다.

Thank you

끝까지 봐주셔서 감사합니다.

CONTACT

+82 010 5920 3386
asm1133@naver.com